Bump Once, Remember Wisely: Calibrated Collision Detection and Forgetting Memory for Aerial Robots Among Invisible Obstacles

Egemen Çevik (150220012), Cumali Sarı (150230038), Ahmet Enes Çiğdem (150220079)
YZV406E Robotics – Term Project Proposal (Round 1), Istanbul Technical University, 10.11.2026

I. Problem Description and Primary Reference Paper

Problem. Autonomous micro aerial vehicles (MAVs) navigate cluttered indoor spaces with LiDAR, assuming that every obstacle is seen. Two common cases break this assumption: (1) *invisible obstacles*—glass walls, doors and windows, transparent curtains and thin wires let laser beams pass through, so the map shows free space and the planner flies into them; and (2) *obstacles that enter unseen*, e.g., from above or below the LiDAR's limited vertical field of view. In both cases a collision cannot be avoided from perception alone. The MAV must therefore (i) *detect* that it collided, (ii) *react* safely before it crashes, and (iii) *remember* the obstacle so that it does not hit it again. Remembering creates a new difficulty: if the obstacle was a person, an object that is later moved, or a door that later opens, a permanent memory turns it into a *phantom wall*. A *false* detection does the same and can make the goal unreachable. The project addresses the suggested topic *Robot Failure Detection and Recovery*. **Our problem** is to detect MAV collisions with a *known* false-alarm rate, and to remember them selectively: keep invisible static obstacles, forget transient ones and false alarms.

Primary reference paper. R. Wang *et al.*, "Air Bumper: A Collision Detection and Reaction Framework for Autonomous MAV Navigation," *IEEE ICRA 2024*, pp. 15735–15741 [1]. It is an original research paper from an approved venue (ICRA) in the required period. A caged MAV with a 3D LiDAR uses only its IMU to: *detect* a collision when any body-axis acceleration exceeds a fixed threshold ($a^* = 20\,\mathrm{m/s^2}$, sliding window $N = 10$); *estimate* its direction and contact point on a spherical cage; *react* immediately with a position setpoint away from the hit, blended with the gradient of the Euclidean distance transform (EDT) of known obstacles; and *remember* it by inserting the contact patch into the volumetric map as a "collision sensor" whose marks are *never cleared* by the LiDAR. Recovery from glass doors is shown in Gazebo, and from glass and stick hits on a real MAV (detection $\approx$0.18 s, recovery $<$1 s, 10 trials).

Why this paper. It is an early framework that combines collision detection, reaction *and* mapping in a standard LiDAR navigation stack using only the IMU. It turns failures into map knowledge ("failure as a sensor"), which is directly a *failure detection and recovery* problem. It is open source [2] and validated in simulation and on hardware. Since its preprint (July 2023) it has been cited by follow-up work on contact-aware flight, e.g., whisker-based tactile navigation in *Nature Communications* [3] and a survey of LiDAR-based aerial vehicles in *IEEE/ASME T-Mech* [4] (7 citing papers on Semantic Scholar, Nov. 2026). Its design choices are simple and explicit (fixed threshold, permanent marks), so their limitations can be analyzed and improved within a term project. Related work handles failures differently: [5] predicts planner failures *before* they happen from LiDAR geometry, and [6] adapts edge costs online from contact feedback; both assume the LiDAR sees every obstacle, which is exactly what fails with glass.

Significance. Indoor MAVs for inspection, warehouse inventory and search and rescue operate around glass facades, windows and partitions, and among people and moving objects. A collision detector whose false-alarm rate is unknown, and a map that never forgets, both cause mission failures: false reactions interrupt flight, and phantom walls block routes. We keep the paper's platform (a caged MAV) and its detect–react–remember pipeline, and improve the two weakest parts: *how* a collision is declared and *how long* it is remembered.

II. Environment, Benchmark, and Dataset

Table I summarizes the environment. Because no public benchmark contains invisible and transient obstacles for MAVs, we build a procedurally generated, seeded **scenario suite**, including the paper's own layout:

- **S1 – Static glass** (60 worlds): indoor rooms and corridors with 1–4 glass walls, glass doors (re-creating the three-door world of [1]) and windows at random positions and heights. Tests detection and memory on invisible static obstacles.
- **S2 – Transient** (45): (a) a glass door that opens 20–120 s after the first contact; (b) a visible panel blocking the shortest route, removed after contact; (c) a pole hitting the cage from below or above the LiDAR FOV (the paper's stick test). Tests the phantom-wall problem.
- **S3 – False-alarm stress** (30): *no collisions*; wind gusts (WindEffects), aggressive stop-and-go and sharp turns, high-speed flight, doubled IMU noise. Measures false alarms per hour.

Worlds are split 50/20/30% into train/calibration/test per set; gust and maneuver profiles unseen during calibration form an out-of-distribution test. Speeds $v_{\max} \in \{0.5, 1.0, 1.5\}$ m/s. We plan $\sim$10 h of collision-free flight for calibration (Sec. IV), a *separate* collision-free test set of $\geq$50 h to verify false-alarm

TABLE I
SOFTWARE AND SIMULATION ENVIRONMENT (ALL FREE).

Component	Choice	Justification
Middleware	ROS 2 Jazzy [7] in Docker	LTS, free; identical setup on every laptop
Autopilot	PX4 SITL [8] via uXRCE-DDS (`px4_msgs`), offboard position/velocity setpoints	Same autopilot as [1]; reaction setpoints go directly to the low-level controller
Simulator	Gazebo Harmonic [9] (PX4 `gz` integration), headless batch runs	Same simulator family as [1]; free ground truth; *WindEffects* for gusts
Vehicle	PX4 x500 quadrotor with a spherical collision cage (radius $l = 0.25\,\text{m}$), 3D LiDAR configured like a Livox Mid-360 ($360^\circ \times 59^\circ$), IMU (250 Hz, noise + bias), cage contact sensor	Mirrors the paper's caged MAV; contact sensor gives ground-truth labels and is *not* used by the detector
Map & planner	Local voxel occupancy map (0.1 m) with EDT [10], A* path search, velocity-limited setpoint follower	Lightweight Python replacement for the paper's GPU mapper and GTO-MPC; exposes cost and speed limits per voxel
Invisible obstacles	Models with collision but no visual geometry	GPU LiDAR renders visuals only, so beams pass through while physics blocks: glass as in [1]
Moving / transient	Joint/trajectory-driven panels and poles, doors opening at random times, objects removed after contact, a "stick" hitting from outside the LiDAR FOV	Creates "remembered but gone" cases and the paper's stick test

TABLE II
LIMITATIONS OF AIR BUMPER [1].

#	Limitation (evidence)	Why it matters
L1	Fixed, hand-tuned threshold on raw acceleration, $a^* = 20\,\text{m/s}^2$ (Sec. IV-A, "experimentally determined"); no false-alarm or miss rates reported	The MAV's *own* aggressive maneuvers and wind gusts also cause spikes, while soft or low-speed hits stay below a^*. The false-alarm rate is unknown and changes with platform, speed and wind
L2	Per-axis test $\lvert a_x\rvert > a^*$ *or* $\lvert a_y\rvert > a^*$ (Sec. IV-A)	A diagonal hit splits between axes: $25\,\text{m/s}^2$ at 45° gives $17.7\,\text{m/s}^2$ per axis and is *missed*
L3	Direction from raw acceleration, $\mathbf{a}_c = -\mathbf{a}$ (Eqs. 1–4)	Commanded acceleration biases the estimated direction, and thus the recovery setpoint and the mapped patch, especially while braking or turning
L4	Permanent, top-priority marks: "the robot remembers all obstacles it ever collides with" (Sec. V-C)	Right for static glass; wrong for people, moved objects, opened doors and *false detections*, which become phantom walls. Detection errors (L1) are made *permanent* by the map
L5	LiDAR evidence at the contact is ignored	Whether the LiDAR *saw* the obstacle separates visible (already mapped, maybe moving) from invisible (must be remembered)
L6	Small, qualitative evaluation: 1 simulated world, 10 real trials	No TPR, false alarms/hour, latency or direction error; no baselines or statistics

rates (headless runs are cheap), and ∼2,000 labeled collisions, which fits into overnight runs with 2–3 parallel containers.

III. CHALLENGES AND LIMITATIONS

Table II lists the limitations of [1] that motivate our project, each with evidence from the paper.

These issues matter because a failure detector is only deployable when its false-alarm and miss rates are known, and because a map that never forgets turns every detection error and every moved object into a lasting navigation failure. The open questions are: *Can MAV collision detection have a guaranteed false-alarm rate instead of a hand-tuned threshold? Can collision memory be selective, keeping invisible static obstacles while forgetting transient ones and its own mistakes?*

IV. PROPOSED METHOD AND NOVEL CONTRIBUTIONS

Baseline reimplementation (B1, not a contribution). We first port Air Bumper to ROS 2 and PX4 SITL: the per-axis detector (a^*, N), the sphere contact model $\mathbf{c}_c = \hat{\mathbf{C}}l$, the recovery setpoint $\mathbf{p}_r = (w\,\mathbf{G}/\|\mathbf{G}\| - (1-w)\hat{\mathbf{C}})R_d$ with $R_d = 2l$ sent directly as a PX4 offboard position setpoint, and permanent, lethal collision voxels in the map.

A. C1: Model-aware, calibrated collision detection (L1–L3)

(a) Residual signal. Instead of raw acceleration we use the *unexpected* acceleration

$$\mathbf{r}(t) = \mathbf{a}^{\text{IMU}}(t) - \hat{\mathbf{a}}(t), \tag{1}$$

where $\hat{\mathbf{a}}$ is the acceleration explained by the MAV's own commands: the commanded velocity/acceleration setpoints are passed through a closed-loop response model of the PX4 controller identified on collision-free flights, and rotated into the body frame with gravity removed. Own braking and turning are thus removed (L1, L3). Model-based residuals are the standard tool for collision detection on manipulators [11]; we bring this idea to IMU-based detection on MAVs.

(b) Features and score. Over a sliding window $W = 0.1\,\text{s}$ we compute the peak residual *norm* $\|\mathbf{r}\|$ in 3D (removes the per-axis issue, L2), the peak jerk, the angular-rate residual, and the setpoint-tracking error (pushing against an obstacle). A small, interpretable logistic regression maps them to a score $s \in [0,1]$; as an ablation, $s = \|\mathbf{r}\|$.

(c) Calibrated threshold. We set the threshold by split conformal calibration [12], [13] on n *collision-free* windows from the calibration worlds:

$$\hat{\tau} = s_{(\lceil (n+1)(1-\alpha) \rceil)}, \tag{2}$$

the $\lceil (n+1)(1-\alpha) \rceil$-th smallest score. Under exchangeability a new collision-free window triggers with probability $\leq \alpha$, so the false-alarm rate is fixed in advance: $\approx \alpha \cdot 3600/W$ per hour for non-overlapping windows. For a target of 1 false alarm/h with $W = 0.1\,\text{s}$, $\alpha \approx 2.8 \cdot 10^{-5}$ and $n \geq 1/\alpha \approx 36{,}000$ windows ($\approx$1 h of flight; we collect ∼10 h). We target $\{1, 5, 20\}$ false alarms/h. Since windows are temporally correlated, we verify the rate empirically on the ≥50 h test set with exact Poisson

(Clopper–Pearson-type) 95% intervals; at 1/h this gives ∼50 expected events.

(d) Residual-based estimation. The direction $\hat{\mathbf{C}}$ (azimuth and polar angle, as in [1]) is computed from $-\mathbf{r}$ instead of $-\mathbf{a}$, so the recovery setpoint and the mapped contact patch are not biased by the MAV's own acceleration (L3).

B. C2: Evidence-aware, forgetting collision memory (L4, L5)

(a) LiDAR evidence. At detection we inspect the LiDAR points within a $\pm 10^\circ$ cone around the contact direction and label the contact *visible* (a return within contact distance $+0.15$ m: already mapped, possibly a person or a moving object), *invisible* (beams pass beyond the contact: the glass case), or *unseen* (direction outside the vertical FOV, e.g., the stick from below).

(b) Decaying belief. Each mark k stores its contact patch, hit count h_k and last-hit time t_k, with belief

$$b_k(t) = \exp\big(-(t - t_k)/\tau_{\text{class}}\big), \tag{3}$$

$\tau_{\text{invisible}} = 600$ s, $\tau_{\text{unseen}} = 60$ s, $\tau_{\text{visible}} = 10$ s (enough to prevent an immediate re-hit; the LiDAR covers the rest). All values are tuned on calibration worlds and studied for sensitivity. A new hit within 0.3 m merges with the mark ($h_k{+}1$, t_k reset): a real glass wall is hit again and stays remembered, while a transient obstacle or a false alarm fades.

(c) Graded costs and speed limits, not walls. A collision voxel gets planning cost $c = c_{\max} b_k$, and is *lethal only if* $h_k \geq 2$ (confirmed); otherwise it is high but passable, so a single unconfirmed mark can never make the goal unreachable. Near decayed or unconfirmed marks the setpoint speed is limited to 0.3 m/s, so re-testing a remembered spot causes at most a gentle cage contact that C1 detects quickly.

Implementation. All nodes are Python (`rclpy`): `imu_residual`, `collision_detector`, `reaction`, `collision_memory`, `voxel_planner`. Memory only changes voxel costs and speed limits, so it can be added to any voxel-based planner. **Expected improvements over [1]:** (1) a false-alarm rate fixed before deployment and verified on test worlds, robust to wind and aggressive maneuvers; (2) detection of diagonal and low-speed contacts; (3) smaller direction errors and better-placed recovery setpoints; (4) no phantom walls with transient obstacles or false alarms, at the cost of a small, measured increase in repeated collisions on static glass (a mark becomes lethal only after a second hit). Whether this trade-off is favorable is tested in RQ3.

V. Research Questions and Evaluation Plan

RQ1 (Detection): Does C1 achieve a higher true-positive rate than Air Bumper's fixed per-axis threshold *at the same false-alarm rate*, across speeds, maneuvers and wind, and does the calibrated rate hold under unseen gust profiles? **RQ2 (Estimation):** How much does residual-based estimation reduce the direction error and improve the recovery distance? **RQ3 (Memory):** Does C2 avoid phantom-wall failures with transient obstacles and false alarms, and how many extra repeated collisions with static invisible obstacles does this cost compared with permanent memory? **RQ4 (System):** Does the full system improve mission success, crashes and time-to-goal over the baselines?

TABLE III
Baselines and ablations.

ID	Detection	Memory	Represents
B0	none (LiDAR-only navigation)	none	Standard MAV stack (crashes, as in [1] Fig. 4b)
B1	Air Bumper [1]: $a^* = 20\,\text{m/s}^2$ and a^* re-tuned on calibration worlds	permanent, lethal	Primary paper
B2	Oracle (cage contact sensor)	permanent	Perfect detection; isolates memory
B3	C1	permanent	C1 alone
B4	Air Bumper	C2	C2 alone
Ours	C1	C2	Full method

TABLE IV
Planned experiments.

#	Experiment	Data	RQ
E1	Offline detection: ROC, TPR at matched false-alarm rate; features vs. $\|\mathbf{r}\|$ vs. raw acceleration; per speed and wind level	S1–S3 test logs	RQ1
E2	Calibration validity: target $\{1, 5, 20\}$/h vs. measured; in-distribution vs. unseen gusts/maneuvers	$\geq$50 h free test set	RQ1
E3	Estimation: raw vs. residual direction; recovery distance	S1, S2	RQ2
E4	Memory (closed loop, oracle detector): permanent vs. decay-only vs. C2	S1, S2	RQ3
E5	Full comparison B0–B4 vs. Ours $\times$ 3 speeds $\times$ 3 seeds	S1–S3 test	RQ4
E6	Ablations/sensitivity: τ, lethal-after-h, speed limit, LiDAR evidence, W	Calibration worlds	RQ3, RQ4

Baselines are listed in Table III. **Metrics.** *Detection* (per collision event, ground truth from the cage contact sensor): TPR, false alarms per hour of collision-free flight (with exact Poisson 95% intervals), empirical vs. target false-alarm rate, detection latency, ROC-AUC. *Estimation:* direction error (deg), post-recovery distance to the obstacle (m). *Memory:* phantom volume (m^3 of marked cost with no real obstacle, integrated over time), repeated collisions (contact within 0.5 m of an earlier one), blocked-goal events. *System:* mission success, crashes (attitude $> 60^\circ$ or uncommanded altitude loss), collisions per run, time-to-goal, node run time ($<$5 ms per cycle).

Experiments are listed in Table IV. **Compute budget.** With ∼41 test worlds, E5 has $6 \times 3 \times 3 \times 41 \approx 2{,}200$ episodes of ∼60 s simulated flight (≈37 h); with PX4 SITL at a real-time factor ≈1 and 3 parallel containers this is ≈12 h of wall-clock time. Fallback: one seed or two speeds for the full grid, keeping all seeds for B1 vs. Ours. **Statistics.** The world is the unit of analysis: 95% bootstrap confidence intervals over worlds, McNemar tests for success/failure and Wilcoxon signed-rank tests for continuous metrics, with Holm correction. All

hyperparameters, including the B1 threshold, are tuned only on calibration worlds. **Threats to validity.** Simulated contacts and IMU are idealized, so we add noise/bias and report results per noise level; the conformal guarantee is approximate under temporal correlation, so we report the empirical rate; sim-to-real transfer is not tested. Implementation risks are checked in week 1: whether collision-only models are invisible to the Gazebo `gpu_lidar` sensor (fallback: a LiDAR visibility mask), and whether the caged x500 survives contacts in SITL without flipping (fallback: tuned contact stiffness and a lower speed range).

VI. Team Roles and Individual Contributions

TABLE V
Team roles.

Member	Role	Responsibilities and deliverables
Egemen Çevik	Simulation, scenarios & experiments	Docker/ROS 2/PX4 SITL/Gazebo bring-up; caged x500 with LiDAR, IMU, contact sensor; invisible-obstacle models; S1–S3 generators; voxel map and planner; episode runner, ground-truth labels; batch runs of E5
Cumali Sarı	Detection & estimation (C1)	Air Bumper detector for B1; controller-response model, residual node, features, logistic regression, conformal calibration, residual-based direction estimation; E1–E3; statistics
Ahmet Enes Çiğdem	Reaction & memory (C2)	Air Bumper recovery control and permanent memory for B1; collision-memory node, LiDAR evidence check, graded costs and speed limits; B3/B4 configs; E4, E6; figures

Roles are given in Table V. Each member writes and presents the part for their own module; contributions are documented by per-member git history and a weekly `CONTRIBUTIONS.md`. **Timeline:** week 1 (11–16 Nov) PX4 SITL + ROS 2 running and verification of the invisible-obstacle model and IMU collision spikes; week 2 S1–S3, voxel planner and B1; week 3 data collection and controller-response model; week 4 C1, E1–E3; week 5 C2, E4; week 6 E5, E6, statistics; week 7 report and slides; presentation on 30.12.2026. **Scope:** minimum = B1 + C1 + E1–E3; target = + C2, E4–E5; stretch = a real-world-like LiDAR model with partial glass returns, and a low-speed contact study.

References

[1] R. Wang, Z. Guo, Y. Chen, X. Wang, and B. M. Chen, "Air Bumper: A collision detection and reaction framework for autonomous MAV navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 15735–15741, doi: 10.1109/ICRA57147.2024.10611410.

[2] R. Wang *et al.*, "Air Bumper," GitHub repository, 2024. [Online]. Available: https://github.com/ryrobotics/air_bumper

[3] C. Ye, G. de Croon, and S. Hamaza, "Whisker-based tactile flight for tiny drones," *Nature Communications*, 2025 (arXiv:2510.03119).

[4] Y. Ren, Y. Cai, H. Li, N. Chen, F. Zhu, L. Yin, F. Kong, R. Li, and F. Zhang, "A survey on LiDAR-based autonomous aerial vehicles," *IEEE/ASME Trans. Mechatronics*, 2025, doi: 10.1109/TMECH.2025.3599633.

[5] N. Mohammad, J. Higgins, and N. Bezzo, "A GP-based robust motion planning framework for agile autonomous robot navigation and recovery in unknown environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 2418–2424, doi: 10.1109/ICRA57147.2024.10610382.

[6] B. He, G. Chen, W. Wang, J. Zhang, C. Fermüller, and Y. Aloimonos, "Interactive-FAR: Interactive, fast and adaptable routing for navigation among movable obstacles in complex unknown environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024, pp. 5402–5409, doi: 10.1109/IROS58592.2024.10802169.

[7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022.

[8] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multi-threaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2015, pp. 6235–6240.

[9] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2004, pp. 2149–2154.

[10] P. F. Felzenszwalb and D. P. Huttenlocher, "Distance transforms of sampled functions," *Theory of Computing*, vol. 8, pp. 415–428, 2012.

[11] S. Haddadin, A. De Luca, and A. Albu-Schäffer, "Robot collisions: A survey on detection, isolation, and identification," *IEEE Trans. Robot.*, vol. 33, no. 6, pp. 1292–1312, 2017.

[12] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. New York, NY, USA: Springer, 2005.

[13] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Found. Trends Mach. Learn.*, vol. 16, no. 4, pp. 494–591, 2023.